TD 1

La POO : pensez en objets !

Premiers pas en **cours inversé** : relisez l'encadré **L'essentiel**, puis réappliquez-le dans l'exercice qui suit.

Nom, prénom ..... Classe ..... 30 min

À VOS SMARTPHONES !

Les paradigmes NOVELCLASS

Le mot-clé self Docstring

1642 EXERCICE 2 — Pascal · la Pascaline
1843 EXERCICE 1 — Lovelace · la Machine analytique
1936 EXERCICE 4 — Turing · la machine universelle
1952 EXERCICE 3 — Hopper · le premier compilateur

Bienvenue dans un nouveau mode de pensée pour la programmation : la **Programmation Orientée Objet** (POO). Fini de donner des instructions pas à pas ! Avec la POO, on organise notre code en créant des « objets » qui interagissent entre eux, comme dans le monde réel.

Définition — Paradigme de programmation

Un **paradigme de programmation** est une manière de structurer et d'écrire un programme. La POO est un paradigme, tout comme la programmation que vous avez utilisée jusqu'à présent.

L'essentiel 1 — Classes et objets : le plan et la voiture

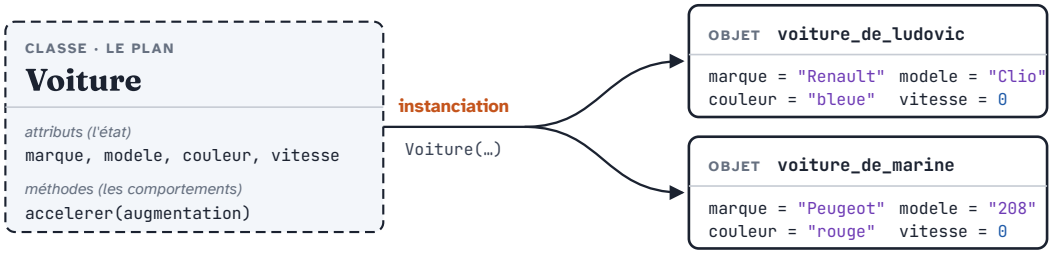
Imaginez que vous voulez acheter une voiture…

- **Classe** = le **plan** : pas une voiture réelle, mais le document qui définit qu'une voiture a des roues, une couleur, un moteur…
- **Objet** = la **voiture concrète** du parking, une **instance** unique du plan. Toutes les voitures suivent le même plan, chacune est un objet.

Que contient une classe ?

- des **attributs** : les **caractéristiques** de l'objet (son état), par exemple `couleur` ou `vitesse` ;
- des **méthodes** : ses **comportements**, des fonctions qui agissent sur l'objet, comme `accelerer()` ou `freiner()`.

Instanciation = créer un objet à partir du plan.



Vous avez dit Eurêka ? La POO, c'est l'art d'organiser son code en regroupant les **données** (les attributs) et les **actions** (les méthodes) qui leur sont liées.

L'essentiel 2 — La POO en Python

```python
class Voiture:                                                  # ◂ le plan
    # Le constructeur : la section "Instructions de fabrication" du plan
    def __init__(self, marque, modele, couleur):                # ◂ constructeur
        # Les attributs
        self.marque = marque                                    # ◂ attributs
        self.modele = modele
        self.couleur = couleur
        self.vitesse = 0 # Attribut avec une valeur par défaut

    # Une méthode
    def accelerer(self, augmentation):                          # ◂ méthode
        self.vitesse += augmentation
        print(f"La voiture roule à {self.vitesse} km/h.")
```

On écrit une classe avec le mot-clé `class`. `__init__` est le **constructeur**, une méthode spéciale appelée **automatiquement** quand on crée un objet.

`self` : désigne **l'objet lui-même**. C'est le premier paramètre de chaque méthode ; il permet d'accéder aux attributs de l'objet (ex. : `self.couleur`).

EXERCICE 1

La Machine Analytique d'Ada Lovelace 1843

Travail sur : vocabulaire et rôles

8 min

Ada Lovelace, pionnière de la programmation, a écrit le premier **algorithme** destiné à la Machine Analytique de Charles Babbage. Observez le code ci-dessous.

```
1  class Machine:
2      def __init__(self, nom, annee):
3          self.nom = nom
4          self.annee = annee
5          self.en_marche = False
6
7      def allumer(self):
8          self.en_marche = True
9
10     def get_etat(self):
11         return self.en_marche
12
13 machine_babbage = Machine("Analytique", 1843)
```

Carte d'identité du programme — questions 1 à 5	
1 Nom de la **classe** définie dans ce programme	
2 Y a-t-il un **objet** ? Si oui, son identifiant (nom)	
3 Type (la classe) de la variable `machine_babbage`	
4 Tous les **attributs** de la classe	
5 Toutes les **méthodes** de la classe	

6 Dans la ligne `self.annee = annee`, que représente `self` ?

7 Quel attribut stocke l'état d'allumage de la machine ? Quel est son type (`int` , `str` , `bool`) ?

8 Expliquez avec vos mots la différence entre `allumer` et `get_etat` : laquelle modifie l'objet, laquelle renvoie une valeur ?

9 Que vaut `machine_babbage.annee` ?

10 Qu'affiche le code ci-dessous ?

```
14 machine_babbage.allumer()
15 print(machine_babbage.en_marche)
```

L'essentiel 3 Créer et utiliser des objets

Créer un objet : on appelle la classe comme une fonction.

```
14 # Création de deux objets (deux voitures)
15 voiture_de_ludovic = Voiture("Renault", "Clio", "bleue")
16 voiture_de_marine = Voiture("Peugeot", "208", "rouge")
```

Accès et modification : on utilise le point « . » pour accéder aux attributs et aux méthodes de l'objet.

```
voiture_de_marine . couleur        voiture_de_ludovic . accelerer(50)
   objet   point  attribut             objet   point   méthode
```

```
17 # Accéder à un attribut
18 print(voiture_de_ludovic.marque) # Affiche "Renault"
19
20 # Modifier un attribut (la rayure !)
21 voiture_de_marine.couleur = "rose" # La voiture de Marine est maintenant rose
22 print(voiture_de_marine.couleur) # Affiche "rose"
23
24 # Appeler une méthode
25 voiture_de_ludovic.accelerer(50) # Affiche "La voiture roule à 50 km/h."
```

Vous avez dit Eurêka ? **Chaque objet est indépendant !** Si vous rayez la voiture de Marine (en modifiant son attribut `couleur`), celle de Ludovic n'est pas affectée.

EXERCICE 2 — La Pascaline de Blaise Pascal XVIIe siècle

Tracer le code et le type de retour

6 min

Blaise Pascal a conçu une machine à calculer mécanique pour aider son père dans son travail, prouvant que les machines pouvaient exécuter des calculs complexes.

```
1  class MachineACalculer:
2      def __init__(self, createur):
3          self.createur = createur
4          self.compteur = 0 # Le cadran d'affichage
5
6      def ajouter(self, nombre):
7          self.compteur += nombre
8
9      def lire_resultat(self):
10         return self.compteur
11
12 pascaline = MachineACalculer("Pascal")
```

Précisez la valeur et le type de chaque variable après l'exécution du code suivant :

```
13 pascaline.ajouter(10)
14 pascaline.ajouter(5)
15 resultat_final = pascaline.lire_resultat()
16 compteur_actuel = pascaline.compteur
```

Après la ligne	12	13	14	15	16
`pascaline.compteur`	0				

Variable	Valeur	Type
1 `resultat_final`		
2 `compteur_actuel`		

EXERCICE 3 — L'interprétation de Grace Hopper années 1950

*Le rôle du point « . » et des méthodes qui modifient **self***

8 min

Grace Hopper, amirale et informaticienne, fut la pionnière du premier **compilateur**, un programme qui **traduit** le code source (lisible par l'humain) en langage machine (exécutable par l'ordinateur). L'une des tâches cruciales du compilateur est de décoder les instructions : identifier à quel objet on s'adresse et quelle action on lui demande.

> **Votre rôle :** vous êtes le compilateur. Votre mission est de décoder les instructions de la classe `Code` et de déterminer leur impact.

```
1  class Code:
2      def __init__(self, lignes):
3          self.lignes = lignes
4          self.est_nettoye = False # État initial : non nettoyé
5
6      def preparer_compilation(self):
7          # La méthode .upper() transforme le texte en MAJUSCULES.
8          self.lignes = self.lignes.upper()
9          self.est_nettoye = True
10
11     def afficher_etat(self):
12         return (self.lignes, self.est_nettoye)
13
14 mon_code = Code("while True:"
15                 "    print('Mouhahah !')")
```

1 La méthode `upper()` est une méthode de quelle classe ? Justifiez.

3 Quel est le type de la valeur renvoyée par la méthode `afficher_etat()` ?

2 Si l'on crée l'objet `mon_code = Code("while true")`, quelle sera la valeur de l'attribut `lignes` après l'appel `mon_code.preparer_compilation()` ?

Vous avez dit Eurêka ? **Vous utilisez déjà des objets !** Les listes (`l.append()`), les dictionnaires (`d.keys()`) ou les chaînes de caractères sont des objets en Python.

EXERCICE 4

Décoder les données universelles d'Alan Turing 1936

Le point : méthode ou accès ?

8 min

En 1936, Alan Turing théorise la **machine universelle**, capable de traiter toute information. En Python, le point (`.`) est l'opérateur universel pour accéder à cette information, qu'elle soit dans une liste, une chaîne ou un nombre flottant.

> **Votre rôle :** déterminez, pour chaque ligne, si le point appelle une action sur un objet (une **méthode**) ou s'il accède à une donnée structurée (un **attribut**).

```
# Données universelles :

ma_liste = [10, 20, 30]
mon_dict = {'a': 1, 'b': 2}
mon_texte = "TURING"
mon_chiffre = 3.14

# Instructions à décoder :

A = ma_liste.append(40)
B = mon_dict.keys()
C = mon_texte.lower()
D = mon_chiffre.is_integer()
E = mon_dict['a']
```

Répondez dans le tableau ci-dessous pour les questions 1 à 4 (entourez oui ou non).

1. Dans quelles lignes (A, B, C, D ou E) le point (`.`) est-il utilisé pour appeler une méthode ?
2. Laquelle des instructions (A, B, C, D ou E) a pour effet de modifier l'objet sur lequel elle est appelée ?
3. Quels sont les types de A, B, C, D et E ?
4. Quelles sont les valeurs de A, B, C, D et E ?

Coup de pouce Testez mentalement chaque ligne : la variable de gauche reçoit ce que **renvoie** l'instruction. Une méthode qui modifie l'objet ne renvoie pas forcément quelque chose !

Instruction	1 Appel de méthode ?	2 Modifie l'objet ?	3 Type	4 Valeur
A `A = ma_liste.append(40)`	oui / non	oui / non		
B `B = mon_dict.keys()`	oui / non	oui / non		
C `C = mon_texte.lower()`	oui / non	oui / non		
D `D = mon_chiffre.is_integer()`	oui / non	oui / non		
E `E = mon_dict['a']`	oui / non	oui / non		

5 Que valent `ma_liste`, `mon_dict`, `mon_texte` et `mon_chiffre` à la fin du code ?

`ma_liste`	`mon_dict`	`mon_texte`	`mon_chiffre`

6 L'instruction E utilise-t-elle le point pour accéder à une valeur ? Expliquez brièvement pourquoi l'utilisation des crochets (`[]`) est une autre manière d'accéder aux données, et pour quels types elle est courante.

Brouillon

Bilan — je sais…

- ☐ Je distingue une **classe** (le plan) d'un **objet** (une instance).
- ☐ Je repère les **attributs** et les **méthodes** d'une classe.
- ☐ Je sais à quoi servent `__init__` et `self`.
- ☐ J'utilise le point : `objet.attribut`, `objet.methode()`.
- ☐ Une méthode peut **modifier** l'objet ou **renvoyer** une valeur.
- ☐ Listes, dictionnaires et chaînes sont des objets.